Relazione esercizio 49 TPSI

Marco Schiavello 4°CI

13/05/2022

INDICE

INDICE 2

IDE utilizzato 3

U.M.L. 3

Classe ClassicBank 3

Classe BankUser 6

Classe Dice 9

Interfaccia Bank 10

Eccezione InvalidDiceProbException 11

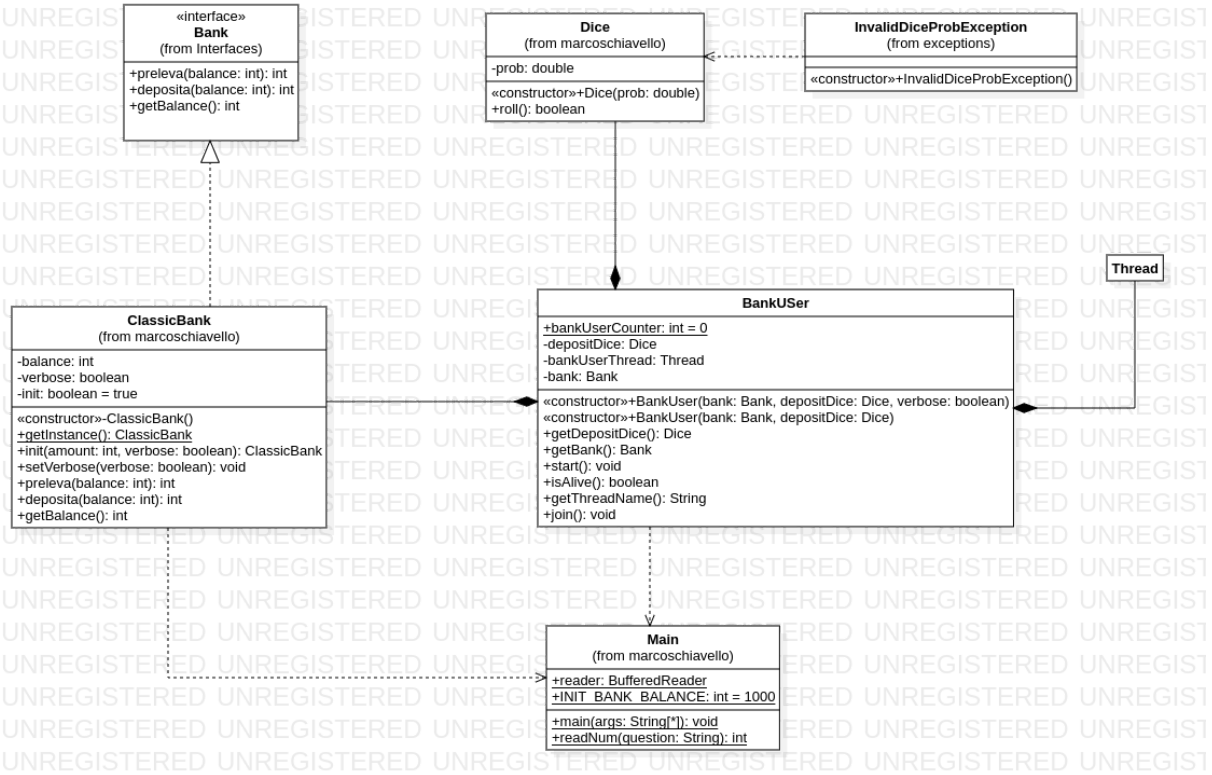
Classe Main 12

Spiegazione generale del progetto 14

IDE utilizzato

Immagini non utilizzare Intellij, imbarazzante, comunque ho usato Intellij.

U.M.L.



Classe ClassicBank

Implementazione di una banca che implementa l'interfaccia Bank essa permette di interagire con il suo conto attraverso dei metodi di deposito e prelievo che sono sincronizzati sull'oggetto banca stesso, per visualizzare il saldo la classe mette a disposizione un metodo getSaldo che sarà sincronizzato anche lui sull'oggetto corrente della banca.
Di questa classe si può avere solo una istanza, questo dato dal fatto che è progettata seguendo il design pattern singleton, che espone un metodo statico per richiamare l'unica istanza della classe e se non presente chiama il costruttore privato.

```
/*******************************************************
*                      ClassicBank                     *
*******************************************************/

package com.marcoschiavello;
```

```
import com.marcoschiavello.Interfaces.Bank;

/**
 * Implementazione di una banca che implementa l'interface {@link
Bank}
 */
public class ClassicBank implements Bank {
    /**
     * Saldo della banca
     */
    private double balance;

    /**
     * Indica se stampare alcuni messagi durante l'esecuzione
     */
    private boolean verbose;

    /**
     * Indica se l'oggetto puo essere ancora inizializzato
     */
    private boolean init = true;

    /**
     * Istanza stastica che sara l'unica della classe come descritto
dal design pattern Singleton
     */
    private static ClassicBank instance = null;

    /**
     * Costruttore privato che construira un oggetto con valori base
     */
    private ClassicBank() {
        this.balance = 0;
        this.verbose = true;
    }

    /**
     * Metodo che ritorna la instanza della classe se presenta
altrimenti chiama il costruttore privato per crearlo, come descritto
dal design pattern Singleton
```

```
     *
     * @return istanza statica della classe
     */
    public static ClassicBank getInstance() {
        if(ClassicBank.instance == null)
            ClassicBank.instance = new ClassicBank();

        return ClassicBank.instance;
    }

    /**
     * Metodo che puo essere chiamato solo una volta e inizializza
con del valori gli attributi della classe
     *
     * @param amount saldo iniziale della banca
     * @param verbose Indica se stampare alcuni messagi durante
l'esecuzione
     * @return istanza statica della classe
     */
    public ClassicBank init(double amount, boolean verbose) {
        if(init) {
            this.balance = amount;
            this.verbose = verbose;
            this.init = false;

            if(verbose)
                System.out.println("La banca ha un saldo iniziale di
" + this.balance + " euro");
        }

        return ClassicBank.instance;
    }

    public void setVerbose(boolean verbose) { this.verbose =
verbose; }

    @Override
    public synchronized double preleva(double balance) {
        return this.balance -= balance;
    }
```

```
    @Override
    public synchronized double deposita(double balance) {
        return this.balance += balance;
    }

    @Override
    public synchronized double getBalance() { return this.balance; }
}
```

Classe BankUser

Classe che descrive un utente della banca che può fare azioni di deposito e prelievo data una banca come argomento del costruttore, questa azioni di deposito e prelievo sono scelte in base al lancio di un dado a 2 che viene preso nel costruttore, infine lo user fa le sue azioni sul conto in modo non bloccante mettendo le azioni su un thread separato.

```
/*******************************************************
*                     BankUser                         *
*******************************************************/

package com.marcoschiavello;

import com.marcoschiavello.Interfaces.Bank;

/**
 * Classe che descrive un utente della banca
 *
 * @author Marco Schiavello
 */
public class BankUser {
    /**
     * Contatore statico che indica l'utente della banca
     */
    public static int bankUserCounter = 0;

    /**
     * Dado usato per decidere se depositare o prelevare
     */
    private Dice depositDice;

    /**
```

```
     *  Thread che eseguirà le azioni di deposito e prelievo
     */
    private Thread bankUserThread;

    /**
     * Banca su cui verrano fatte le azioni di deposito e di
prelievo
     */
    private Bank bank;

    /**
     * Costruttore completo che crea uno user per una determinata
banca
     *
     * @param bank banca di cui si vuola fare lo user
     * @param depositDice dado che verra usato per decidere se
prelevare o depositare
     * @param verbose
     */
    public BankUser(Bank bank, Dice depositDice, boolean verbose) {
        this.depositDice = depositDice;
        this.bank = bank;

        this.bankUserThread = new Thread(() -> {
            double balanceAfterAction = 1;
            while(balanceAfterAction > 0) {
                int amount = (int) Math.floor(500 + (Math.random() *
250));
                boolean isDeposit = depositDice.roll();

                balanceAfterAction = isDeposit ?
bank.deposita(amount) : bank.preleva(amount);
                if(verbose)

System.out.println(Thread.currentThread().getName() + " ha " +
(isDeposit ? "depositato" : "prelevato")  + " " + amount + " euro,
ne rimangono " + balanceAfterAction + " sulla banca");
            }
            if(verbose)
                System.out.println(Thread.currentThread().getName()
+ " ha finito");
```

```
        }, "BankUserThread-" + BankUser.bankUserCounter);

        BankUser.bankUserCounter ++;
    }

    /**
     * Construttore accorciato che sottointende che verbose è a true
     *
     * @param bank banca di cui si vuola fare lo user
     * @param depositDice dado che verra usato per decidere se
prelevare o depositare
     */
    public BankUser(Bank bank, Dice depositDice) { this(bank,
depositDice, true); }

    public Dice getDepositDice() { return this.depositDice; }
    public Bank getBank() { return this.bank; }

    /**
     * matodo wrapper di {@link Thread#start()}
     */
    public void start() { this.bankUserThread.start(); }


    /**
     * matodo wrapper di {@link Thread#isAlive()}
     */
    public boolean isAlive() { return this.bankUserThread.isAlive();
}

    /**
     * matodo wrapper di {@link Thread#getName()}
     */
    public String getThreadName() { return
this.bankUserThread.getName(); }

    /**
     * matodo wrapper di {@link Thread#join()}
     */
    public void join() throws InterruptedException {
this.bankUserThread.join(); }
```

```
}
```

Classe Dice

Classe che identifica un dado a due facce che prevede una probabilità che esca una faccia rispetto a un'altra presa nel costruttore, utilizzato per decidere quale azione far fare a un utente della banca. Questo dado a due facce permette di decidere la probabilità che esca una faccia rispetto ad un altra.

```
/*******************************************************
*                     Dice                             *
*******************************************************/

package com.marcoschiavello;

import com.marcoschiavello.exceptions.InvalidDiceProbException;

/**
 * Classe che identifica un dado a due faccie che prevede una
probabilita che esca una faccia rispetto a un altra presa nel
costruttore
 *
 * @author Marco Schiavello
 */
public class Dice {
    /**
     * Probabilita che esca una faccia rispetto a un altra
     */
    private double prob;

    /**
     * Costruttore completo che prende la probabilita per dado e lo
crea
     *
     * @param prob probabilita che esca una faccia rispetto a un
altra
     * @throws InvalidDiceProbException
     */
    public Dice(double prob) {
```

```
        if(prob < 0 || prob > 1)
            throw new InvalidDiceProbException();

        this.prob = prob;
    }

    /**
     * Metodo che dice che dice e uscita la facia con la probabilita
indicata nel costruttore, qursto sara indicato con il
     * ritorno true altriemnto se esce false sara uscita la faccia
con la probailita rimanente
     *
     * @return risultato del lancio del dado
     */
    public boolean roll() { return Math.random() < this.prob; }
}
```

Interfaccia Bank

Interfaccia che impone i metodi base che deve avere una banca per interagire con gli utenti di essa.

```
/*******************************************************
*                     Bank                             *
*******************************************************/
package com.marcoschiavello.Interfaces;

/**
 * Interfaccia che descrive i metodi che devono essere implementati
dentro banca che imlpementa questa interfacia
 *
 * @author Marco Schiavello
 */
public interface Bank {
    /**
     * Metodo che prevela dal saldo della banca un ammontare preso
come parametro
     *
     * @param balance ammontare da prelevare
     * @return il saldo rimanente sul conto dopo l'operazione
```

```
     */
    double preleva(double balance);

    /**
     * Metodo che deposita nella banca un ammontare preso come
parametro
     *
     * @param balance ammontare da depositare
     * @return saldo del conto dopo l'operazione
     */
    double deposita(double balance);

    /**
     * Metodo per prendere il saldo del conto
     *
     * @return saldo del conto
     */
    double getBalance();
}
```

Eccezione InvalidDiceProbException

Eccezione usata dalla classe Dice quando viene messo nel costruttore una probabilità non valida.

```
/*******************************************************
*               InvalidDiceProbException               *
*******************************************************/

package com.marcoschiavello.exceptions;

/**
 * Eccazione che viene lanciata quando viene data una probabilita
invalida al costruttore di {@link com.marcoschiavello.Dice}
 *
 * @author Marco Schiavello
 */
public class InvalidDiceProbException extends RuntimeException {
    /**
```

```
     * Costruttore che chiama il costruttore di {@link
RuntimeException} passandogli il messaggio di errore
     */
    public InvalidDiceProbException() {
        super("Invalid probabolity, it must be between 0 and 1");
    }
}
```

Classe Main

La classe main in questo programma si occupa di gestire la creazione e avvio dei thread della banca e delle persone(che sono messi in un arrayList).
Alla fine dell'esecuzione di tutti i thread(usando join) stampa il saldo finale della banca e finisce.

```
/*******************************************************
*                          Main                        *
*******************************************************/

package com.marcoschiavello;

import com.marcoschiavello.Interfaces.Bank;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static BufferedReader reader = new BufferedReader(new
InputStreamReader(System.in));
    public static int INIT_BANK_BALANCE = 1000;

    public static void main(String[] args) throws
InterruptedException{
        List<BankUser> bankUsers = new ArrayList<>(); // array di
user della banca
        int numOfUser = readNum("Quanti utenti dalla banca vuoi");
// chiedo in input il numero di bank user che agiranno
parallelamente sulla banca
```

```
        Bank bank =
ClassicBank.getInstance().init(INIT_BANK_BALANCE, true); // creao la
banca con il metodo statico come descritto dal pattern singleton e
poi chiamo un metodo init per dargli valori di base

        // creo un nuovi user della banca e li aggiungo a una lista
in segito li faccio partire
        for(int i = 0; i < numOfUser; i++) {
            bankUsers.add(new BankUser(bank, new Dice(0.5)));
            bankUsers.get(i).start();
        }

        // aspetto che tutti gli user mandino in negativo il saldo
della banca
        for(BankUser bankuser : bankUsers)
            bankuser.join();

        System.out.println("La banca e riamsta con un saldo di " +
bank.getBalance() + " euro"); // stampo il saldo finale
    }

    /**
     * Metodo statico per leggere un intero positivo
     * @param question domanda da porre per avere un valore
dall'utente
     * @return valore immesso dal utente
     */
    public static int readNum(String question) {
        try {
            int num;
            System.out.print(question + ": ");

            while ((num = Integer.valueOf(Main.reader.readLine())) <
1) {
                System.out.print(question + ": ");
            }

            return num;
        } catch (IOException e) {
            e.printStackTrace();
```

```
            return -1;
        }
    }
}
```

Spiegazione generale del progetto

Questo programma aveva come scopo di mettere in risalto l'aspetto delle race condition dove operazioni non atomiche vengono interrotte da altre istruzioni compromettendo così la caratteristica di un algoritmo di essere deterministico.
In questo programma per risolvere il problema delle race condition abbiamo usato la keyword synchronized che usa il lock intrinseco degli oggetti nello heap per far eseguire parti di codice solo da un thread alla volta, togliendo il parallelismo ma rendendo l'algoritmo di nuovo deterministico.